

DISTRIBUTED COMPUTING ENGINE // v3.x

APACHE SPARK

Lightning-Fast Unified Analytics Engine for Big Data

100×

faster than Hadoop MR

4

unified components

PB

scale data processing

WHAT IS APACHE SPARK?

DEFINITION

Apache Spark is an **open-source unified analytics engine** for large-scale data processing. It provides high-level APIs in **Java, Scala, Python & R**, with optimized engine supporting general computation graphs.

In-Memory Processing

Fault Tolerant (RDD lineage)

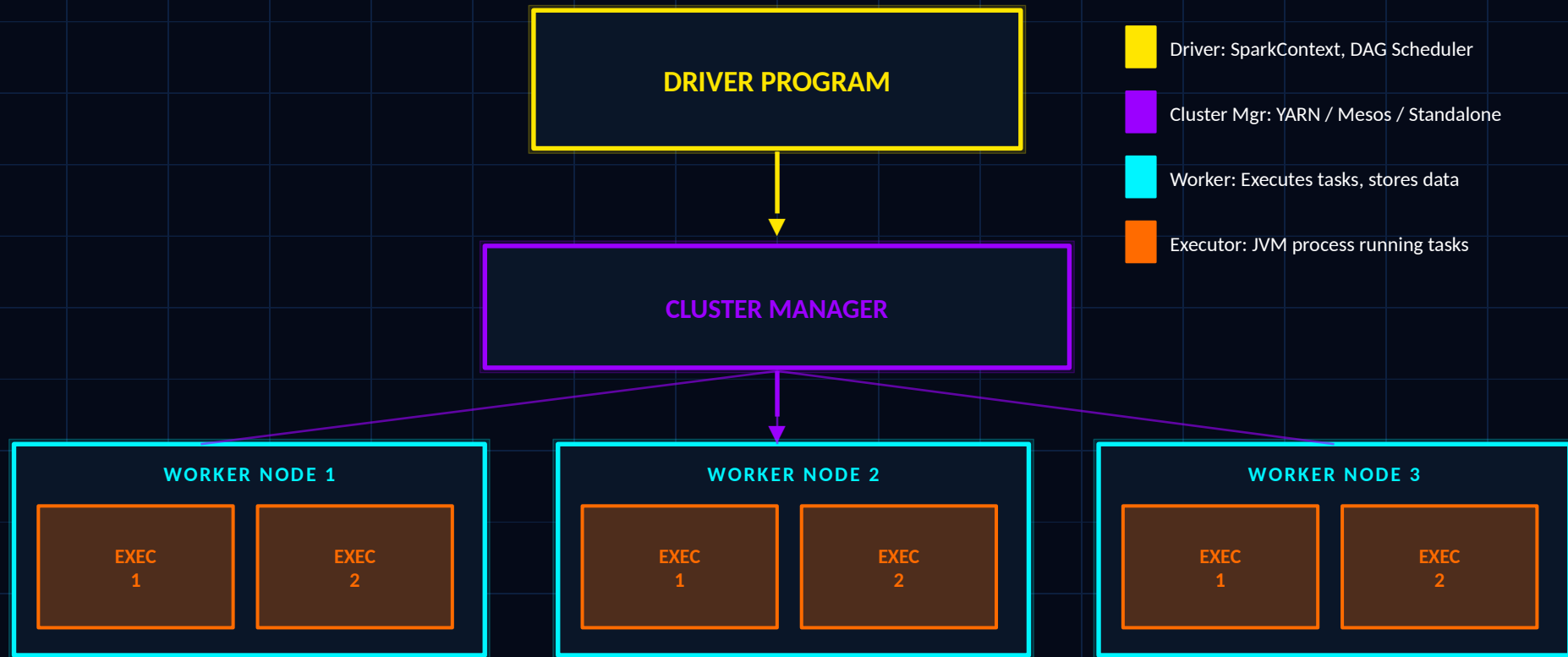
Lazy Evaluation

Distributed by design

SPARK vs HADOOP MapReduce

FEATURE	SPARK	HADOOP
Processing	In-Memory	Disk I/O
Speed	100× faster	Baseline
Ease of Use	High-level APIs	Verbose
Streaming	✓ Native	✗ Limited
ML Support	✓ MLlib built-in	✗ External
Graph	✓ GraphX	✗ External

SPARK CLUSTER ARCHITECTURE



RDD — RESILIENT DISTRIBUTED DATASET

WHAT IS AN RDD?

An RDD is the fundamental data structure of Spark — an immutable, distributed collection of objects partitioned across nodes that can be processed in parallel.

R**RESILIENT**

Fault-tolerant via
lineage graph

D**DISTRIBUTED**

Partitioned across
cluster nodes

D**DATASET**

Collection of any
record type

RDD OPERATIONS

TRANSFORMATIONS (lazy — return new RDD)**map()****groupByKey()****filter()****reduceByKey()****flatMap()****join()****ACTIONS** (eager — trigger computation)**collect()****first()****count()****reduce()****take(n)****saveAsTextFile()**

DATAFRAME & DATASET API

DATAFRAME

A distributed collection of data organized into named columns — like a table in a relational DB or a pandas DataFrame.

- Schema-aware
- Catalyst optimizer
- Python / Scala / Java / R
- SQL query support
- Tungsten execution engine

DATASET (Scala / Java only)

A strongly-typed, object-oriented API. Combines the benefits of RDD (compile-time type safety) with the optimizations of DataFrame.

- Type-safe at compile time
- Encoder-based serialization
- Lambda & OOP support
- Best of RDD + DataFrame

EVOLUTION: **RDD** → **DataFrame** → **Dataset** | Use DataFrame / Dataset for new code — better optimizations!

SPARK ECOSYSTEM — 4 COMPONENTS



SPARK SQL

Structured Data Processing



Run SQL queries on DataFrames



JDBC/ODBC connectivity



Hive compatibility



Catalyst query optimizer



SPARK STREAMING

Real-Time Data Processing



Micro-batch streaming (DStream)



Structured Streaming (preferred)



Kafka / Kinesis / Flume sources



Windowed aggregations



MLlib

Machine Learning Library



Classification, Regression



Clustering, Collaborative Filter



Pipeline API



Distributed training



GRAPHX

Graph Computation



Graph-parallel computations



PageRank, Triangle Count



Pregel API



ETL + graph algorithms

PYSPARK — CODE PATTERNS

TRANSFORMATIONS

```
from pyspark.sql import SparkSession
spark = SparkSession.builder\
    .appName('MyApp').getOrCreate()

df = spark.read.csv('data.csv',
                    header=True, inferSchema=True)

# Filter + Select
result = df.filter(df['age'] > 25)\
    .select('name', 'salary')\
    .groupBy('dept')\
    .agg({'salary': 'avg'})
```

SQL + ACTIONS

```
# Register as SQL temp view
df.createOrReplaceTempView('emp')

sql_df = spark.sql("""
    SELECT dept,
           AVG(salary) as avg_sal,
           COUNT(*) as headcount

    FROM emp
    WHERE age > 25
    GROUP BY dept
""")

sql_df.show() # ← ACTION
```

LAZY EVALUATION & DAG EXECUTION

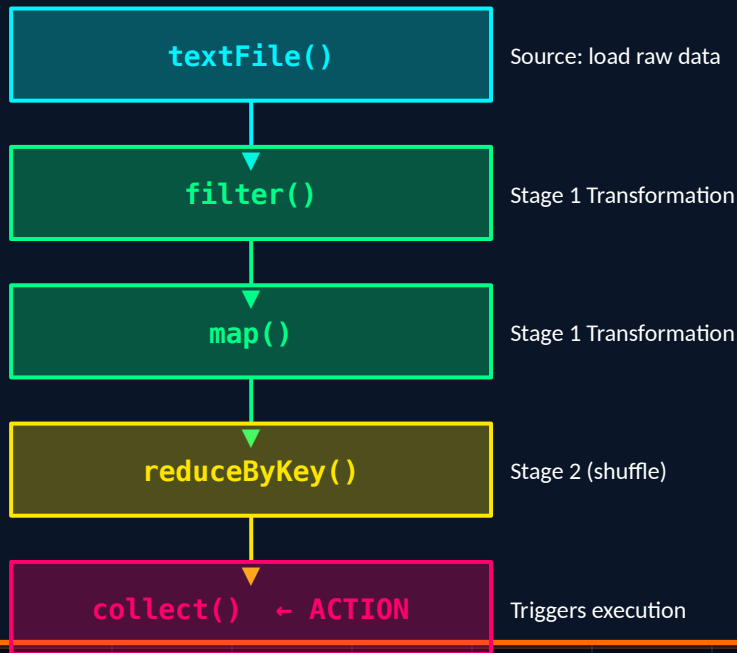
LAZY EVALUATION

Transformations are NOT executed immediately. Spark builds a logical plan (DAG) and only executes it when an Action is called. This allows Spark to optimize the entire pipeline.

WHY LAZY EVALUATION?

- 1 Pipeline Optimization**
Whole-stage code generation
- 2 Avoid Unnecessary Work**
Skip unused data partitions
- 3 Fault Recovery**
Recompute only failed partitions

DAG — DIRECTED ACYCLIC GRAPH



SPARK QUICK REFERENCE

KEY CONCEPTS

SparkContext

Entry point to Spark cluster (legacy)

SparkSession

Unified entry point (use this!)

RDD

Low-level distributed data abstraction

DataFrame

Tabular data with schema & SQL

Dataset

Typed DataFrame (Scala/Java)

Partition

Chunk of data on a single node

Shuffle

Data redistribution across partitions

Broadcast Var

Read-only var shared to all nodes

SPARK WORKFLOW

1

INIT

Create SparkSession with config

2

INGEST

Read CSV / Parquet / JSON / DB

3

TRANSFORM

Apply filter, map, groupBy, join

4

OPTIMIZE

Cache hot DataFrames, repartition

5

EXECUTE

Trigger action: show / collect / write

6

SINK

Write to Parquet, Delta, DB, Kafka